# PackerとServerspecではじめるインフラ自動化

2018/9/1合同勉強会 in 大都会岡山 -2018 Summer-

大中浩行(@setoazusa)

この作品は クリエイティブ・コモンズ 表示 4.0 国際 ライセンスの下に提供されています。

Copyright 2018 Hiroyuki Onaka

# こんにちわ！

- 大中浩行(@setoazusa / せとあず)
- 横浜から来ました ※明日から東京都民です
- TDDBC/XPJUG/JJUG/錬金術MeetUp
- 「Mackerelではじめるお手軽Webサービス監視」(インプレスR&D)

Copyright 2018 Hiroyuki Onaka

# よろしくお願いします

CC BY

# こんなことにお悩みではないですか？

- 再起動するとアプリケーションに接続できなくなる
  (SELinuxとかFirewalldとか)
- サーバーごとにロケールとかTimeZoneとかまちまち
- 気づいたらディスクを食い尽くしてるログ。
  ローテーションはどうした
- その不具合は以前の案件でも対応したやつじゃないですか？
- えっ脆弱性対応で要アップデート？

Copyright 2018 Hiroyuki Onaka

# そこで自動化！Infrastructure As Codeですよ！

なんですが…

Copyright 2018 Hiroyuki Onaka

# インフラ自動化あるある

- スクリプトの結果を確認するには本番環境一発勝負
- スクリプトの適用時はEdit and Pray
- dry-runってどこまであてになるの？

Copyright 2018 Hiroyuki Onaka

# 今日する話

- VagrantとPackerの併用による、自動化の生産性向上
- PackerとServerspecの組み合わせによるによるインフラCI
- AWS前提です。

Copyright 2018 Hiroyuki Onaka

# VagrantとPacker

Copyright 2018 Hiroyuki Onaka

# Vagrant

- いわずとしれた仮想環境構築ツール
- ローカルでの開発環境として使用

Copyright 2018 Hiroyuki Onaka

# Vagrantのポイント

- ベースのイメージをしっかり管理する
- ローカルのVMイメージとAWS上のAMIイメージの基点を可能な限り揃える

Copyright 2018 Hiroyuki Onaka

# Packerの使用

- VMイメージの構築ツール
- 同一テンプレートからVagrant(VirtualBox)とAWS向けのイメージを作成することにより、構成の統一を行う

Copyright 2018 Hiroyuki Onaka

# BoxCutterによるベースイメージの作成

- https://github.com/boxcutter
- https://github.com/boxcutter/centos をフォークして、AWS向けのamiも作成できるようにしたものが https://github.com/azusa/centos

Copyright 2018 Hiroyuki Onaka

# Vagrant(VirtualBox)向けイメージの作成

```
packer build -var-file=centos7.json
```

centos.json

Copyright 2018 Hiroyuki Onaka

# AMIイメージの作成

```
packer build jira-ami.json
```

Copyright 2018 Hiroyuki Onaka

# このベースイメージをもとにして、アプリケーションのプロビジョニングを行います。

- プロビジョニングツールはChefです。
- なんでChefかって？そりゃこのレシピのメンテナンス始まったのが2014年だからですよ

Copyright 2018 Hiroyuki Onaka

# どうプロビジョニングするか

- VagrantにせよPackerにせよ、各種プロビジョニングツールとの連携機能があります。
- しかし、VagrantとPackerでそれぞれプロビジョニングツールとの連携を行うと、異なるインターフェースを用いてプロビジョニングを行うことになり、結果の不整合やプロビジョニング失敗などの問題が起こります。

Copyright 2018 Hiroyuki Onaka

# ツール間ででのプロビジョニングのインターフェースの統一

VagrantとPackerでインターフェースを統一するため、シェルスクリプトを一層かまして、両方が同じロジックでプロビジョニングされるようにします。

Copyright 2018 Hiroyuki Onaka

# Vagrantfile

```
config.vm.define :local, primary: true do |local|
    local.vm.hostname="jira.local"
    local.vm.box = "centos7.5-1804-1"
    local.vm.network :private_network, ip: "192.168.3.11"
local.vm.provision :shell, :path => "provisioning-vagrant.sh", :args => ["software-local"]
  end
```

Copyright 2018 Hiroyuki Onaka

# Packerの設定ファイル

```
"provisioners": [
    {
      "type": "file",
      "source": ".",
      "destination": "/tmp/"
    },
(略)
```

```
"execute_command": "echo
'vagrant' | {{.Vars}} sudo -E
-S bash '{{.Path}}'",
      "scripts": [
        "provisioning-packer.sh",
        "spec.sh"
      ],
      "type": "shell",
      "pause_before": "10s"
    }
  ],
```

Copyright 2018 Hiroyuki Onaka

# シェルスクリプトの実装(抜粋)

```
curl -L https://www.opscode.com/chef/install.sh | bash

set +e

service systemctl stop jira

set -e

chef-client -z -c ${CURRENT}/solo.rb -j ${CURRENT}/nodes/${1}.json -N ${1}
```

Copyright 2018 Hiroyuki Onaka

# Ansibleの場合だと、こうなります

```
yum -y install ansible

CURRENT=$(cd $(dirname $0) && pwd)

cd $CURRENT && PYTHONUNBUFFERED=1 ANSIBLE_FORCE_COLOR=true
ansible-playbook --limit="default" --inventory-file=localhost -v
provision/localhost.yml
```

Copyright 2018 Hiroyuki Onaka

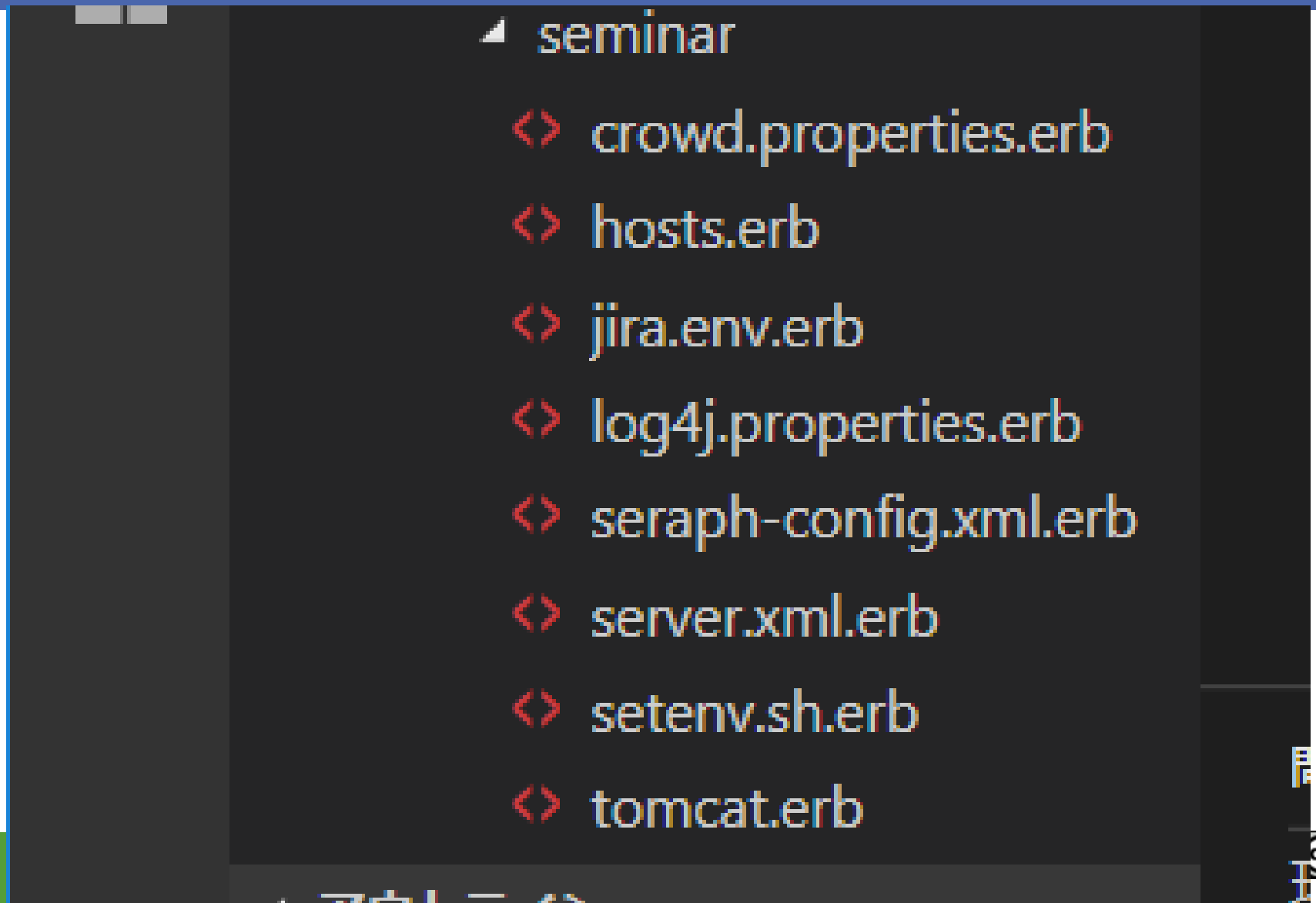
# 環境ごとのイメージだし分け

AMIによるイメージ作成の場合、環境ごとの設定値の切り替えは起動時の環境変数の指定で行って、全ての環境で統一されたイメージを使用するのが原則です。

Twelve-Factor Appってやつですね

Copyright 2018 Hiroyuki Onaka

# だが、そうもいかない場合がある



# どうするか

- 起動時のユーザーデータのスクリプトで設定ファイルの切り替えを行う
- 環境ごとにイメージを作成する
→ユーザーデーターのスクリプトが複雑になることを避けたかったため、環境ごとにイメージを作成する方法を採用しています。

Copyright 2018 Hiroyuki Onaka

# 現状でのユーザーデータがこれくらいあるので…

```
set -x

device_id_data="/dev/xvdb"
mount_point_data="/mnt/atlassian/jira/data"

parted -l |grep -A 6 ${device_id_data} |grep xfs
FORMATED=$?
```

```
# ファイルシステムが作成されてなかったら
## ファイルシステムの作成
if [ $FORMATED -ne 0 ]; then
mkfs -t xfs ${device_id_data}
fi

## マウントポイントの作成
echo "${device_id_data} ${mount_point_data} xfs noatime,defaults 0 0" >> /etc/fstab
```

Copyright 2018 Hiroyuki Onaka

# Chef二段構え

設定ファイルの切り替えを行う部分の構造を単純化するため、デフォルト設定を使用したイメージを作成し、Serverspecによるテストを行った後、設定ファイルの切り替えを行うPackerを再度実行しています。

Copyright 2018 Hiroyuki Onaka

# Serverspecによるインフラ CI

Copyright 2018 Hiroyuki Onaka

# Serverspec

base_spec.rb - Visual Studio Code

ファイル(F) 編集(E) 選択(S) 表示(V) 移動(G) デバッグ(D) タスク(T) ヘルプ(H)

base_spec.rb

```ruby
require 'spec_helper'


describe command("date") do
    its(:stdout) { should contain "JST" }
end


describe command("timedatectl |grep \"Time zone\" |grep \"Asia/Tokyo\" ") do
        its(:exit_status) { should eq 0 }
end


describe file("/etc/selinux/config") do
    its(:content) { should contain "SELINUX=disabled" }
end
```

master* 0 0 行 21、列 1 スペース: 4 UTF-8 LF Ruby

Copyright 2018 Hiroyuki Onaka

# ServerspecによるインフラCIの実行パターン(1)

- Packerによるイメージ作成時に、Serverspecを実行するか
- 作成したイメージを基に仮想マシンを起動し、その仮想マシン上でServerspecを実行するか

Copyright 2018 Hiroyuki Onaka

# Serverspecによるインフラ CIの実行パターン(2)

- Serverspecをホスト上(local)で実行するか
- ホストにSSH接続して実行するか

Copyright 2018 Hiroyuki Onaka

手順を複雑化しないため、Packerによるイメージ作成時に、ホスト上でServerspecを実行する方法を採用しています。

Copyright 2018 Hiroyuki Onaka

# Serverspecの起動に使うRubyをどうするか

- Chefのomibusインストーラー(/opt/chef/embedded)配下のRubyバイナリーを使用して、Serverspecを実行しています。

Copyright 2018 Hiroyuki Onaka

# Serverspecの起動スクリプト(spec.sh)

```
set -x
export PATH=/opt/chef/embedded/bin:$PATH
export BACKEND_LOCAL="local"
CURRENT=$(cd $(dirname $0) && pwd)
cd $CURRENT
sudo -E /opt/chef/embedded/bin/bundle install
bundle exec rake spec
RET=$?
systemctl stop jira
rm -rf /mnt/atlassian/jira/data/*
rm -rf /tmp/*
exit $RET
```

Copyright 2018 Hiroyuki Onaka

# Packerの設定内でのServerspec設定(抜粋)

```
"execute_command": "echo 'vagrant' |
{{.Vars}} sudo -E -S bash '{{.Path}}'",
"scripts": [
"provisioning-packer.sh",
"spec.sh"
],
```

Copyright 2018 Hiroyuki Onaka

# まとめ

Copyright 2018 Hiroyuki Onaka

# Pact+ServerspecによるインフラCIの成果

- バージョンアップの所要時間2時間→10分
- 自動化によるプロセスの標準化
- Serverspecによるレシピの信頼性向上

Copyright 2018 Hiroyuki Onaka

# 10/8 技術書典よろしくお願いします！

- う-02
  「ふぃーるどの一つ」
- 今日お話しした内容を同人誌にします！
- 通販も行います！
  fieldnotes.booth.pm

Copyright 2018 Hiroyuki Onaka

# ありがとうございました！

- 大中浩行(Onaka,Hiroyuki)
- @setoazusa
- http://hiroyuki.fieldnotes.jp/

Copyright 2018 Hiroyuki Onaka